Actividad 7: Mini juego Piedra, Papel o Tijera con persistencia de datos (struct + fstream)

E.E.S.T. N.º 1 "Eduardo Ader" — Vicente López

Laboratorio de Programación (LPR) — 5.º año, 3.º [A/B]

Docente: Mansilla Muñoz York Elías

Versión: 1.0.0 — 2026

Integrantes y roles

Integrante	Rol
Federico Mattia	Creador del informe y administrador del repositorio
Maximo Orue	Programador principal
Ulises Ferrari	Tester y responsable de las capturas de evidencia
Lucas del Pino	Documentación y revisión final

Repositorios

GitHub: https://github.com/mattia4567/piedrapapelotijera.git

Resumen

El trabajo consiste en desarrollar un mini juego de Piedra, Papel o Tijera en C++ para consola. El usuario compite contra una elección aleatoria de la computadora. Cada partida se representa mediante una estructura y se guarda en un archivo de texto para que los registros permanezcan disponibles después de cerrar el programa.

Palabras clave: C++, struct, fstream, persistencia, archivos de texto.

Introducción

La actividad busca diferenciar la memoria RAM, que se utiliza durante la ejecución del programa, del almacenamiento secundario, que permite conservar información en archivos. Para demostrarlo, el programa registra cada partida en historial_partidas.txt y

ofrece una opción para leer el historial.

Objetivos

- Agrupar los datos de cada partida en una estructura Partida.
- Escribir registros con ofstream e ios::app sin sobrescribir partidas previas.
- Leer el historial mediante ifstream.
- Validar las opciones ingresadas por el usuario y mostrar el resultado de cada partida.

Desarrollo

La estructura Partida contiene el ID, nombre del jugador, elección del jugador, elección de la IA y resultado. La función jugarPartida solicita el nombre, recibe una elección válida, genera una elección aleatoria y aplica las reglas del juego. La función guardarEnDisco agrega los datos al archivo y leerHistorialDisco los recupera para mostrarlos en la consola.

Persistencia de datos

El archivo se abre con ofstream en modo ios::app. Este modo agrega la nueva línea al final del archivo y conserva los registros anteriores. Para consultar los datos se utiliza ifstream, que lee los campos separados por comas. El archivo se encuentra en src/historial_partidas.txt.

Compilación y ejecución

```
g++ -std=c++11 src/main.cpp -o src/piedrapapelotijera.exe
.\src\piedrapapelotijera.exe
```

Los comandos deben ejecutarse desde la carpeta raíz del proyecto.

Resultados y evidencias

Después de compilar, se debe jugar al menos una partida, revisar el archivo de historial, cerrar y volver a ejecutar el programa y comprobar que las partidas anteriores sigan disponibles. Adjuntar las capturas reales en la carpeta capturas/: jugada_consola.png y archivo_disco.png.

Cuestionario técnico de control

1. RAM vs. disco: ¿Por qué el historial sigue disponible al reabrir el programa?

Porque las partidas se escriben en un archivo del disco, que no desaparece cuando termina el proceso. Las variables de la RAM sí se pierden al cerrar el programa, pero el archivo historial_partidas.txt permanece.

2. Uso de ios::app: ¿Qué ocurriría si se omitiera?

Al abrir un ofstream sin ios::app ni otro modo de anexado, normalmente se crea o trunca el archivo para escritura. Eso podría borrar el historial anterior y dejar solamente los datos nuevos.

3. Búfer y cin.ignore(): ¿Por qué se usa antes de getline()?

La lectura anterior con cin >> deja el Enter pendiente en el búfer. cin.ignore() descarta ese carácter para que getline() pueda esperar y leer el nombre completo.

4. Agrupación con struct: ¿Qué ventajas ofrece?

Permite reunir en una sola entidad los cinco datos relacionados con una partida. Las funciones reciben un objeto Partida, lo que hace el código más organizado y evita pasar muchas variables sueltas.

5. Aplicación al P.I.A.: ¿Qué datos podrían guardarse?

Se podrían guardar eventos del prototipo, por ejemplo fecha y hora de una lectura, valores de sensores, cambios de estado, movimientos de stock o registros de usuarios. El archivo funcionaría como una bitácora para revisar qué ocurrió.

Conclusión

La actividad permite aplicar estructuras, funciones y manejo de archivos en un caso práctico. La escritura con ios::app y la lectura con ifstream demuestran que los datos pueden conservarse entre ejecuciones. Las pruebas finales deben verificarse mediante la ejecución del programa y las capturas solicitadas.

Referencias

cppreference.com. (s. f.). *Input/output library: File stream classes and open modes*. https://en.cppreference.com/w/cpp/header/fstream

E.E.S.T. N.º 1 "Eduardo Ader". (2026). *Actividad 7: Mini juego "Piedra, Papel o Tijera" con persistencia de datos (struct + fstream)* [Material de clase]. Laboratorio de Programación.